

PHP : Bien débuter avec PHP

Ce tutorial a pour but d'introduire les débutants à PHP dans les meilleures conditions possibles. Il est fondamental lorsqu'on débute de partir sur de bonnes bases, car c'est le moment où l'on prend ces marques, et donc où l'on prend ses habitudes, bonnes ou mauvaises...

Ce tutorial fera le tour de ce qu'il faut connaître de PHP, de sa syntaxe la plus basique jusqu'aux structures de contrôle, en passant par son histoire et ses différentes installations. Vous y apprendrez l'essentiel de ce que vous devez savoir dans l'optique de vos débuts en PHP.

Des exemples illustreront toutes les nouvelles notions et nous en arriverons à la fin de ce tutorial à un exemple complet.

1. **PHP : Qu'est-ce que c'est ?**
2. **Installer PHP : EasyPHP et WAMP5 ?**
3. **La syntaxe : premier code PHP**
4. **Les variables et le dollar : \$**
5. **Superglobales et formulaires**
6. **Structures de contrôle : if, else, else if, switch, while, for**
7. **Les fonctions utilisateurs**
8. **include() et require()**
9. **Programme utilisant tout ce qui a été appris dans le tutorial**

1. **PHP : Qu'est-ce que c'est ?**

PHP (abréviation de PHP Hypertext PreProcessor), c'est un langage de programmation Web dynamique, c'est-à-dire qu'une même page Web écrite en PHP pourra être différente à chaque appel, en fonction de différents paramètres. Paramètres qui peuvent être ceux que vous désirez. De plus, PHP est un langage Open Source, c'est-à-dire qu'il est totalement gratuit et que quiconque peut en modifier le code source s'il le souhaite.

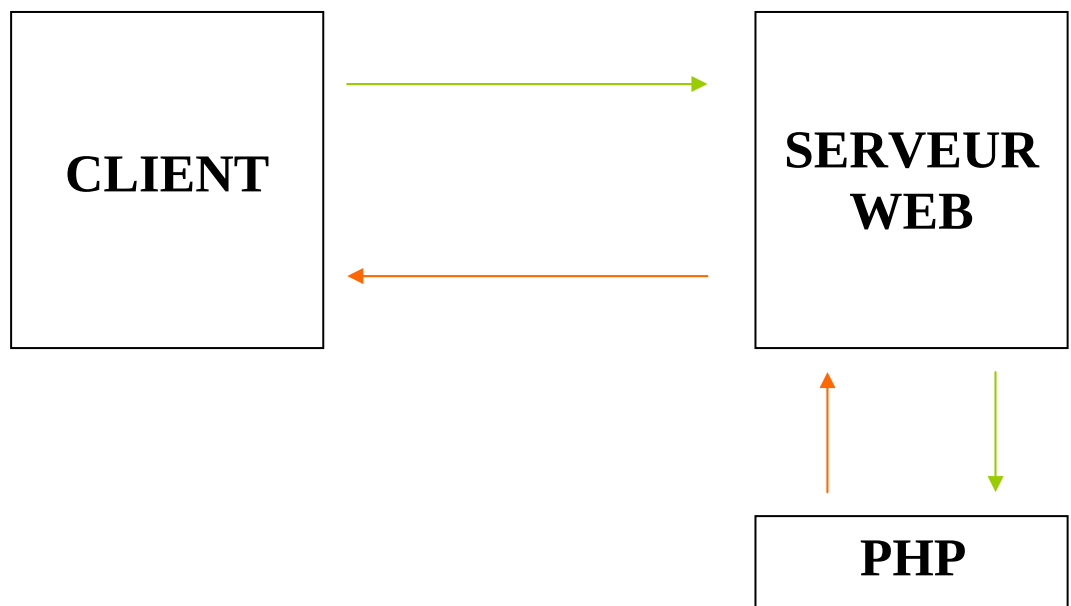
PHP a été créé en 1995 par Rasmus Lerdorf. Son premier nom était *Personal Home Page Tools* et il s'agissait à l'époque d'une bibliothèque de scripts Perl. C'est ensuite devenu **PHP/FI** (*Personal Home Page / Forms Interpreter*) car, chose rare à cette époque, **PHP/FI** gérait les formulaires. Puis **PHP/FI 2** est sorti en 1997. C'est alors que **PHP 3** (pour *PHP Hypertext PreProcessor*) fut publié en 1998. C'est cette version 3 qui a fait mondialement connaître PHP. En 2000, c'est **PHP 4** qui a vu le jour. Cette version est aujourd'hui la plus répandue dans le monde. C'est d'ailleurs sur celle-ci que nous travaillerons. Enfin, c'est cette année qu'est sorti la version 5 de PHP. **PHP 5** est cependant

trop peu répandu sur le net (et notamment sur les hébergeurs) pour qu'il soit judicieux de commencer par cette mouture.

C'est donc sur **PHP 4** que nous travaillerons. Notez que tout ce qui sera abordé dans ce tutorial peut fonctionner avec **PHP 3**, moyennant parfois une syntaxe quelque peu différente.

2. Installer PHP : *EasyPHP* ou *WAMP5* ?

Avant de programmer en PHP, il faut l'installer. Eh oui ! PHP est un langage dynamique et on ne peut donc pas visualiser une page PHP comme une page HTML en l'ouvrant simplement dans un navigateur. PHP est en fait géré « côté serveur », c'est ce qui fait sa dynamicité. Le mieux est d'expliquer son fonctionnement par un schéma :



Le client demande la page au serveur Web. Celui-ci demande à PHP (en fait au *Zend Engine*) d'interpréter le code PHP de la page. Une fois cette action terminée, PHP renvoie le code généré au serveur, qui le rajoute au code statique de la page (c'est-à-dire le code HTML). Enfin le serveur envoie la page (désormais constituée uniquement de code HTML) au client.

Maintenant que vous savez comment fonctionne PHP, il vous faut installer un serveur Web « en local », c'est-à-dire sur votre PC, autrement il faudrait envoyer la page sur votre serveur Web (votre hébergeur) pour la tester. Grâce au serveur Web installé en local, vous pouvez tester une page dynamique PHP simplement en tapant l'adresse dans votre navigateur.

La plus simple façon d'installer PHP est d'utiliser une installation automatique de Apache/PHP/MySQL (MySQL est un Système de Gestion de Base de Données, ou SGBD, mais cela ne fait pas parti de ce tutorial). Sous Windows,

il en existe deux : EasyPHP (actuellement dans sa version 1.7) et WAMP5, qui propose PHP 5. Je vous conseille pour ce dernier de télécharger le plug-in PHP 4 qui vous permet de faire tourner votre serveur avec PHP 4 et non PHP 5 (vous pourrez toujours intervertir à tout instant entre les versions 4 et 5).

Ces deux logiciels sont disponibles aux adresses :

<http://www.easyphp.com/>

et

<http://www.wamp5.org/>

3. La syntaxe : premier code PHP

Maintenant que vous pouvez exécuter vos scripts PHP, il est temps d'en venir à la programmation elle-même. Voici comment intégrer du PHP dans une page HTML :

```
<HTML>
<HEAD>
    <title>Ma Page HTML avec du PHP</title>
</HEAD>
<BODY>
    <?
        echo "Ceci est un texte produit avec PHP";
    ?>
</BODY>
</HTML>
```

Vous aurez remarquer les balises **<?** et **?>**. Ce sont les balises qui délimitent le script PHP. Il en existe en fait plusieurs formes :

Ouverture	Fermeture
<?	?>
<%	%>
<?php	?>
<script language="php">	</script>

Toutes ces balises sont équivalentes, mais attention de les utiliser par paires : **%>** ne fonctionne pas si le bloc PHP a été ouvert avec **<?** !

Dans ce bloc PHP, il n'y a qu'une seule instruction :

```
echo "Ceci est un texte produit avec PHP.";
```

L'instruction *echo* permet d'afficher le texte situé entre les guillemets. Le point-virgule (;) en fin de ligne signale la fin de l'instruction. Il est OBLIGATOIRE. Si vous l'oubliez, vous obtiendrez une erreur du type :

Parse error: parse error, unexpected T_ECHO , expecting ',' or ';' in
c:\www\index.php on line 34

Le code présenté ci-dessus affichera « Ceci est un texte produit avec PHP. » en haut à gauche de la page Web. Ce n'est bien sûr qu'un exemple très basique qui nous permet d'introduire la syntaxe de PHP.

4. Les variables et le dollar : \$

Un contenu dynamique ne peut être dynamique sans variables. Ce sont elles qui contiennent les informations qui doivent changer d'un appel à un autre de la page. Pour les utiliser, il suffit de faire précéder leurs noms d'un dollar (\$) :

```
$ma_premiere_variable;
```

La variable \$ma_premiere_variable est alors déclarée, et on peut lui assigner une valeur. Cette valeur peut être :

- Un entier :
Un nombre entier, positif ou négatif.
- Un décimal :
Un nombre à virgule, positif ou négatif.
- Une chaîne de caractères :
Une suite de caractères.
- Un booléen :
Une variable qui ne prend que deux valeurs : *true* (vrai) et *false* (faux).

Pour assigner une valeur à une variable, il suffit d'utiliser l'opérateur « = » :

```
$ma_premiere_variable = 15;  
ou  
$ma_premiere_variable = 124.55;  
ou  
$ma_premiere_variable = "Ceci est une chaîne de caractères";  
ou  
$ma_premiere_variable = true ;
```

Le type de variable (entier (int), décimal (float ou double), chaîne de caractères (string), booléen (bool)) est automatiquement assigné.

Voici un exemple d'utilisation de variables :

```
<?  
    $nom = "Glabouni";  
    $prenom = "Jean";  
  
    echo 'Bonjour, je m'appelle '.$prenom.' '.$nom.'.';  
?>
```

Ici, il est intéressant de remarquer la concaténation des chaînes de caractères. Elle se fait grâce au point (.). On aurait pu écrire :

```
echo "Bonjour, je m'appelle $prenom $nom.";
```

Il existe une différence importante entre les « *simple quotes* » (' et ') et les « *double quotes* » (" et ") dans une instruction *echo* : entre « doubles quotes », PHP analyse la présence de variables à interpréter (afin d'en afficher la valeur et non le nom), alors qu'entre « simples quotes » il ne s'en occupe pas et affiche le nom de la variable (par exemple « \$variables »). Ainsi on n'aurait pas obtenu le même résultat en écrivant :

```
echo 'Bonjour, je m\'appelle $prenom $nom.';
```

Il s'avère que les « doubles quotes » nécessitent plus de ressources puisqu'il y a analyse de la chaîne, il est donc préférable de n'utiliser ces « doubles quotes » que si l'on a des variables à interpréter. C'est une des bonnes habitudes à prendre. Les deux écritures à préférer sont donc :

```
echo 'Bonjour, je m\'appelle '.$prenom.' '.$nom.'';
```

et

```
echo "Bonjour, je m'appelle $prenom $nom.";
```

Les opérations arithmétiques entre les entiers et les décimaux sont :

Opérateur arithmétique	Signification
+	addition
-	soustraction
*	multiplication
/	division
%	modulo

5. Superglobales et formulaires

Il ne suffit pas de manier les variables pour obtenir un contenu dynamique, il faut également laisser au visiteur la possibilité d'interagir avec le site. Pour cela, on a recours aux formulaires. Ces formulaires sont des formulaires HTML, dont le code se présente comme ceci :

```
<form name= "form1" method= "GET" action "page2.php">
<input type="text" name="text1">
<input type="text" name="text2">
<input type="submit" value="Envoyer">
</form>
```

La méthode utilisée ici est GET. Il existe une seconde méthode : POST.

Voyons comment récupérer les variables text1 et text2 passée à la page « page2.php » :

Pour la méthode GET :

```
<?
    echo 'text1 vaut '._GET['text1'].' et text2 vaut
    '._GET['text2'].';
?>
```

Pour la méthode POST :

```
<?
    echo 'text1 vaut '._POST['text1'].' et text2 vaut
    '._POST['text2'].';
?>
```

On récupère les variables grâce aux syntaxes `$_GET['variable']` et `$_POST['variable']`.

Pour être certain que la variable a bien été envoyée par la méthode voulue, on peut utiliser les codes :

```
if(!empty($_GET['variable'])) {}
if(isset($_GET['variable'])) {}
```

(L'instruction **if** sera vue au chapitre suivant).

6. Structures de contrôle : if, else, else if, switch, while, for

Ce que l'on a vu pour l'instant nous permet d'obtenir des variables différentes à chaque appel de la page. Cependant, on ne peut pas encore changer le comportement de la page en fonction de ces variables. Pour cela, il faut utiliser les structures de contrôle. Il en existe plusieurs formes : les conditions et les boucles.

▪ Les conditions :

○ Les instructions **if/else** :

L'instruction **if** vérifie une condition et exécute le code qui se trouve dans le « bloc if » si celle-ci est vraie. Sinon, c'est le code du « bloc else » qui est exécuté :

```
if(condition)
{
    //code à exécuter si la condition est vraie
}
```

```

else
{
    //code à exécuter sinon
}

```

//s'il n'y a pas de « bloc else » et si la condition n'est pas vérifiée, le programme saute ici. Le programme saute tout de même ici si la condition est vérifiée, mais après avoir exécuter le « bloc if ».

Il existe une instruction **else if**(condition) qui, placée après le « bloc if », permet de vérifier une seconde condition si la première n'est pas vérifiée. Elle peut se combiner avec l'instruction **else** :

```

if(condition1) { //code à exécuter si la condition1 est vraie }
else if(condition2) { //code à exécuter si la condition2 est vraie }
else if(condition3) { //code à exécuter si la condition3 est vraie }
else { //code à exécuter sinon }

```

o Les instructions **switch/case** :

L'instruction **switch** prend la valeur d'une variable et la compare à de multiples valeurs (des « cas »). Si l'un des cas correspond, le bloc qui y est associé est exécuté :

```

switch($variable)
{
    case 1://code à exécuter si $variable vaut 1;break;
    case 2://code à exécuter si $variable vaut 2;break;
    case 3://code à exécuter si $variable vaut 3;break;
    case 'chaîne'://code à exécuter si $variable vaut 'chaîne';break;
    default: //code à exécuter quand $variable ne correspond à aucun cas;break;
}

```

Les blocs sont terminés par « break; » pour ne pas exécuter les éventuels blocs situés après.

▪ Les boucles :

o L'instruction **while** :

La boucle **while** exécute le code du « bloc while » tant que la condition est vraie :

```

while(condition)
{
    //code à exécuter tant que la condition est vraie
}

```

- o L'instruction **for** :

Une boucle **for** répète une instruction *n* fois :

```

for($i=0;$i<n;$i++)
{
    //code à exécuter tant que la condition est vraie
}

```

n doit être un nombre fixé. La variable *\$i* est en fait un compteur, qui est incrémenté (c'est-à-dire qu'on lui rajoute 1) à chaque passage. La première instruction des parenthèses est l'initialisation de la variable. Elle ne s'exécute qu'une fois, au début de la boucle (et non à chaque passage). La seconde est la condition qui, tant qu'elle est vraie, fait que le bloc est exécuté. La troisième instruction est l'instruction qui se fait en fin de bloc. Elle s'exécute à chaque passage.

Pour sortir d'une boucle sans rendre la condition fausse, il faut utiliser **break**. Pour arrêter un seul passage d'une boucle et recommencer au passage suivant (surtout utile dans une boucle **for**), on utilise **continue**.

Voici un exemple de code qui utilise les instructions **if/else** et **for** :

//On suppose que la page reçoit la variable \$age par la méthode GET

```

<?
if($_GET['age']<18)
{
    echo 'Vous êtes mineur';
}
else if($_GET['age']>=18 && $_GET['age']<65)
{
    echo 'Vous êtes majeur et n\'êtes pas encore à la retraite';
}
else
{
    echo 'Vous êtes à la retraite';
}
?>

```

Le && signifie « ET ». Il existe également d'autres opérateurs logiques :

Opérateur logique	Signification
&&	ET (les deux conditions sont vraies)
	OU (l'une des condition est vraie)
!	FAUX (la condition est fausse)

//Calcul de la factorielle d'un nombre entier \$nb passé par la méthode GET*

<?

```
$factorielle=1;
for($i=1;$i<=$_GET["nb"];$i++)
{
    $factorielle=$factorielle*$i;
}
```

?>

**Si vous ne connaissez pas la factorielle (notée « ! »), voici deux exemples : 5!
= 5*4*3*2*1 = 120, 8! = 8*7*6*5*4*3*2*1 = 40320*

7. Les fonctions utilisateurs

Il existe ne nombreuses fonctions en PHP, mais ces fonctions (dites natives) ne suffisent pas parfois pour faire du code simple. Pour cela, on peut faire des fonctions utilisateurs :

```
function MaFonction(argument1, argument2, ...)
{
    //instructions de la fonction
}
```

Grâce à ces fonctions, on peut nettement simplifier un code, en appelant une seule fonction pour tout un bloc. Bien sûr les fonctions ne comportent pas d'intérêt si on ne l'utilise qu'une seule fois. Elles prennent tout leur intérêt lorsqu'on exécute le même bloc d'instructions plusieurs fois. Par exemple, on peut se trouver dans la situation suivante :

Les arguments sont remplacés lors de l'appel de la fonction par les valeurs que l'on passe à la fonction. Voici un exemple :

<?

```
function carre($nombre)
{
    return $nombre*$nombre ;
}
```

```
$nb=11;
$nbcarre = carre($nb);
```

```
echo 'Le carré de '.$nb.' est '.$nbcarre.'';
?>
```

Le mot-clé `return` indique que la fonction retourne une valeur (ici la valeur de `$nombre*$nombre`). Cela signifie qu'on peut récupérer cette valeur dans une variable (ici on la récupère dans `$nbcarre`).

La fonction « `carre` », quand on l'appelle avec comme argument `$nb`, retourne la valeur `$nb*$nb`. Elle a remplacé `$nombre` par `$nb`.

8. `include()` et `require()`

On peut rencontrer, par exemple quand on utilise beaucoup de fonctions utilisateurs et qu'on les retrouve dans plusieurs pages, le besoin d'inclure une page dans une autre. Cela est possible grâce à deux fonction : `include()` et `require()` :

```
include('mapage.php');
et
require('mapage.php');
```

Ces deux fonctions incluent la page « `mapage.php` », et sont identiques à une différence près : `include()` ne renvoie qu'un « `warning` » en cas d'erreur tandis que `require()` renvoie une erreur. Si vous vous en tenez à une utilisation simple, cette différence ne pose aucun problème.

9. Programme utilisant tout ce qui a été appris dans le tutorial

Voici un programme réalisé avec ce qui a été appris dans ce tutorial :

```
//fonctions.php
<?
function carre($nombre)
{
    return $nombre*$nombre;
}

function racine($nombre)
{
    return sqrt($nombre);
}
?>

//index.php
<?
$titre = "Carrés et Racines Carrées";
echo "<HEAD><title>".$titre."</title></HEAD>";
?>
<form name="form1" method="POST" action="affiche.php">
<input type="text" name="nb_carre" >
```

```
Afficher tout ?<input type="checkbox" name="affichetout">
<input type="submit" value="calculer tous les carrés" >
</form>
<form name="form1" method="GET" action="affiche.php">
<input type="text" name="nb_racine">
Afficher tout ?<input type="checkbox" name="affichetout">
<input type="submit" value="calculer toutes les racines carrées">
</form>
```

//affiche.php

<?

```
include('fonctions.php');
if(isset($_POST['nb_carre']))
{
    $nb=$_POST['nb_carre'];
    switch($_POST['affichetout'])
    {
        case false: echo "carré de $nb : ".carre($nb); break;
        case true:for($i=1;$i<=$nb;$i++)
        {
            echo "carré de $i : ".carre($i)."<br>";
        }
        break;
    }
}
else if(isset($_GET['nb_racine']))
{
    $nb=$_GET['nb_racine'];
    switch($_GET['affichetout'])
    {
        case false:echo "racine carrée de $nb : ".racine($nb); break;
        case true:for($i=1;$i<=$nb;$i++)
        {
            echo "racine carrée de $i : ".racine($i)."<br>";
        }
        break;
    }
}
else
{
    echo 'Vous n'avez pas fourni de nombre !';
}
?>
```

Le programme a deux comportements :

- Si l'utilisateur entre un nombre dans le premier formulaire et l'envoie, le programme affiche les carrés des nombres entiers de 1 jusqu'au nombre entré par l'utilisateur.
- S'il utilise le second formulaire, ce sont les racines qui s'affichent.

L'utilisateur peut choisir si le programme affiche tous les carrés (ou toutes les racines carrées) depuis 1 ou uniquement le carré (ou la racine carrée) du nombre qu'il a entré en cochant la case « Afficher tout ? ».

Tutorial « PHP : Bien débuter avec PHP » écrit par tHE_fLAmMinG_mOE le 06/11/2004.